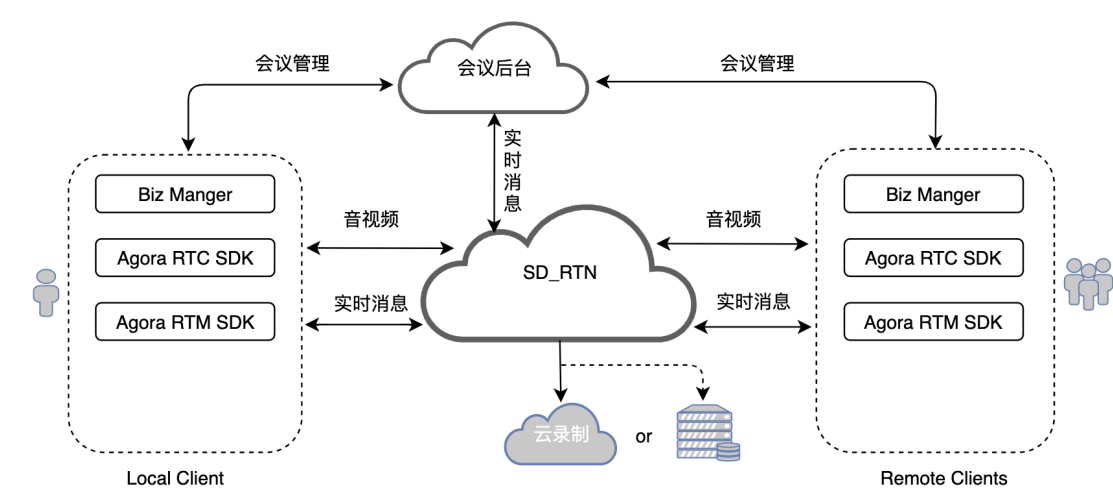


会议最佳实践-20250113

简述

本文编写目的，为使用Agora SDK集成视频会议场景的客户参考的解决方案

业务架构



会议场景

会议场景主要功能清单

基本功能	协作功能	会议管控	AI能力	硬件适配	第三方对接
• 音/视频通话 • 互动直播 • 会议录制 • 3A处理 • web/小程序入会 • 通讯录对接 • 加入会议 • 预约会议 • 单屏显示50路 • 3000人大会	• 屏幕共享（共享窗口/共享应用/共享区域） • 互动白板 • 实时消息/IM • 文档协作 • 会议弹幕 • 无线投屏	• 会议邀请 • 角色/权限管理 • 会议密码 • 视图切换 • 全体静音 • 移出会议 • 分组讨论 • 控制举手发言 • 会议质量监控	• 美颜 • 虚拟背景 • AI降噪 • 暗光增强 • 视频降噪 • 啸叫监测 • 鉴黄 • 实时转写/转译	• 麦克风/扬声器 • 摄像头 • 会议平板/商显 • 会议专用终端 • 机顶盒 • AR/VR设备 • 智能家居类 • 智能穿戴类 • 智能车载类	• PSTN电话 • Volte • Sip会议终端 • 融合CDN • RTSP码流

会议优化

1. sdk核心能力

新一代音频技术智能引擎：凤鸣AI引擎

AI降噪

不依赖硬件算法
有效抑制突发噪声

AI回声消除

实现复杂场景下的非线性回声抑制
和近端人声保真

凤鸣美音 虚拟声卡

一部手机即可获得
媲美专业主播的音质效果

有效提升娱乐社交场景 用户通话时长

▼ **13%** ▲
超短 (≤30s) 通话 中长 (≥1min) 通话

某游戏开黑场景客户，开启AI降噪后

有效解决智慧教室 突发噪声及回声困扰

“声网AI降噪和回声消除的效果大大超出我们的预期。未来，我们覆盖的16,000所学校的录播设备都将基于声网的音频3A进行全面升级。”

——某头部智慧教室厂商 CEO

超级画质：人更美、物更真，体验更流畅

极致高清

人更美，画面更沉浸

美颜悦色

更自然的美颜效果

低码高清

更低带宽，更高画质

镜头律动

更生动的画面效果

丝滑流畅

进出、切换频道0感知

某客户升级超级画质后，频道内用户留存时长增加，打赏率大幅提升

↑ **19%**
停留超过 30s 观众数

↑ **14%**
观众次均观看时长

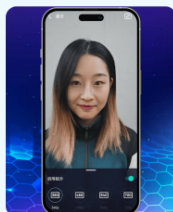
↑ **30%**
观众人均观看时长

大幅提升
观众打赏率

数据来源：东南亚头部泛娱乐平台

极致高清：人更美，画面更沉浸

人更美



端侧实时超分
实现画质跨级提升，
全面迎接 1080P 高清视野



视频增强
暗光增强+色彩增强+
视频降噪 多维度画质增强



显微镜级细节捕捉
AI 感知处理算法
更懂你的美

画面更沉浸



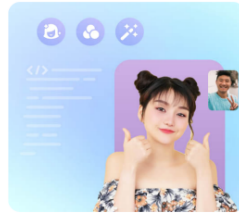
4K60fps 超高清屏幕共享
高清的同时保障流畅，
带来游戏直播大片体验

美颜悦色：AI超级美颜，人更美，更自然



RTC 美颜最佳实践

超低性能消耗，
支持热门出海区域720P开播



场景化美颜 API 显著降低开发门槛

300+ 行代码减至 10 行
开发时间由 7 天缩短至 3 小时



高精准度不同人种五官贴合

高稳定定位追踪，
不抖动无延时

低码高清：更低带宽，更高画质



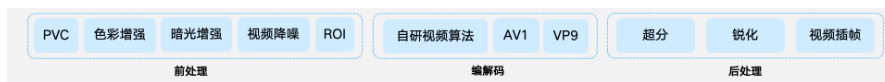
全行业最优码率编码传输方案，码率
最高 **下降70%**。最低 500K 即可实
现 720P 视频流畅播放体验



同等码率下，大幅提升直播画质。能够
实现人眼感知画质从**360p提升至**
480p，或540p提升至720p



码率下降，不影响人眼主观感受。例如
通过编码控制减少游戏中草地画面的码
率，而整体画面的**人眼感知不变**



丝滑流畅：进出、切换频道用户0感知

极速出图，无感切屏，极大降低频道切换的响应延迟

100ms

首帧出图时间

<500ms

1v1 视频连通平均耗时

97%

秒开率

利用先进的编码技术和传输算法，整体**优化网络传输**和**LastMile传输策略**，进一步降低首帧出图时间和切频道出图时间，并且有效提升码流爬升速度，用户对图像切换零感知。

超强的网络适应能力：极致抗弱网

保障全球音视频互动一致性体验



码率&帧率
智能调节



端到端全链路
优化算法



优秀的
抖动缓冲机制



多维度
网络估计模型



多路径传输
网络动态切换

80% 丢包情况下，音视频通话流畅

超低卡顿，国内视频平均卡顿率小于 1.1%，海外视频平均卡顿率小于 2%

支持30+平台开发框架、30000+终端机型

多平台架构兼容

兼容 iOS、Android、Windows、macOS、Web、鸿蒙OS、小程序RTOS、Electron、Flutter、React Native、Unity、Cocos等共计超过30个平台开发框架，

保证全平台一致性体验



适配 30000+ 移动终端

- 针对中低端机型适配并进行性能优化，可以很好地覆盖国内下沉市场以及东南亚、两印、非洲等不发达区域机型
- 适配各类智能终端、会议终端、监控前端等设备，场景化覆盖面广



2. 弱网优化

优化策略包括如下：

- 播放器Buffer
- 丢帧策略（优先丢P帧，其次I帧，最后音频）
- 自适应码率算法
- 双向链路优化
- 音频FEC冗余算法（20%丢包率）

自适应码率

在弱网情况下，另外一种靠谱的策略是自适应码率算法，通过设置码率降级为多个档次，这样，当网络不好的情况下，通过降低码率进行预测，如果码率降低后，还不够buffer缓冲，那么继续降低一个档次，是一个循环探测的过程，如果再次降级一个档次后，发现buffer缓冲足够了，那么说明当前网络能够适应这个码率，因此就会采取当前码率。同理，升档也是一样的。

3. VQC改进

- 针对不同输入视频内容，推荐更合理的编码参数，使得产生的视频主观体验更好；
- 适应不同网络环境，自适应调整分辨率和帧率，保障视频主观体验；
- 适用于不同用户偏好设置，可灵活调整画质优先偏好或者流畅度优先偏好。

4. sdk abr能力优化

通过音视频策略调整，保障多终端、多网络、多布局切换场景下，画质、音质、性能、带宽最佳平衡，体现声网音视频领域的核心优势。

- 体现弱网/低端机优势。

2. 体现音质/画质优势。

推流profile：大小流

- 小型会议，人数<25（默认）：
 1. pc端摄像头流：720p（大流）、360p（小流）
 2. 移动端摄像头流：480p（大流）、240p（小流）
 3. pc端屏幕共享流：1080p（大流）、360p（小流）
 4. 移动端屏幕共享流：720p（大流）、360p（小流）
- 大会，人数>=25：
 1. pc端摄像头流：720p（大流）、180p（小流）
 2. 移动端摄像头流：480p（大流）、180p（小流）
 3. pc端屏幕共享流：1080p（大流）、360p（小流）
 4. 移动端屏幕共享流：720p（大流）、360p（小流）

推流profile：abr

1. pc端摄像头流：720p（大流）、360p（中流）、180p（小流）、90p（弱流）
2. 移动端摄像头流：480p（大流）、240p（中流）、144p（小流）
3. pc端屏幕共享流：1080p（大流）、360p（小流）
4. 移动端屏幕共享流：720p（大流）、360p（小流）

推流设备性能自适应：

1. 接收端请求多流时：根据设备性能情况按需开启。
2. 运行中性能变差：同步降低多流profile（画质优先）。

上行弱网自适应：

1. 接收端请求多流时：根据发送端网络情况按需开启。
2. 运行中网络变差：同步降低多流profile（画质优先）。

推荐设置

1. 音频配置

profile+scenario+channel-profile

	channelProfile	profile	scenario
windows/macOS	CHANNEL_PROFILE_LIVE_BROADCASTING = 1	AUDIO_PROFILE_SPEECH_STANDARD = 1	AUDIO_SCENARIO_MEETING = 8
Android/iOS	CHANNEL_PROFILE_LIVE_BROADCASTING = 1	AUDIO_PROFILE_SPEECH_STANDARD = 1	AUDIO_SCENARIO_CHATROOM = 5
Android大屏	CHANNEL_PROFILE_LIVE_BROADCASTING = 1	AUDIO_PROFILE_SPEECH_STANDARD = 1	AUDIO_SCENARIO_CHATROOM = 5

参数设置

音频编解码

- 提高抗弱网能力
 - {"che.audio.pad_fec.offset": 1}
 - {"che.audio.pad_fec.bitrate": 10000}
 - {"che.audio.pad_fec.num": 1}

高音质

一般高音质

- opus+高码率：{"che.audio.custom_bitrate":128000}

高级高音质

- HEAAC
{"che.audio.custom_payload_type":78} : //48000, 1920, 2, 32000
{"che.audio.custom_bitrate":128000}
- 3A: 如果可以, 关闭3A会有帮助

2. 视频配置

推荐：分辨率，码率

		帧率/fps	码率/kbps
PC	1280*720	30	1150
	960*540	30	1000
		15	700
	640*360	15	500
	480*270	15	250
移动	640*360	15	700
	480*270	15	150

		发送			订阅	
	人数	5	10	> 14 • 50	演讲者	宫格
PC	大流	1280*720	1280*720	960*540	主：大流	小流
	小流	640*360	640*360	480*270	其他：小流	
移动	大流	960*540	960*540	640*360		
	小流	640*360	640*360	480*270		

参数设置

编解码

编码器选择

- che.video.a264_encode:true ==>推荐使用a264, false→x264

软硬件编解码

- 硬件编码: che.hardware_encoding: 1 (硬)
- 硬件解码: che.hardware_decoding: 1 (硬)
- 小流硬件编码: rtc.video.low_stream_enable_hw_encoder: true

弱网对抗

打开CC2

- che.video.enableNewCC:true

关闭带外fec

- che.video.fec_outside_bw_ratio:0

集成参考

1. 设备管理

PC端视频设备/视频流变化回调（以Windows为例）：

onLocalVideoStateChanged[视频基础功能](#) | [文档中心](#) | [声网 \(shengwang.cn\)](#)
onVideoDeviceStateChanged [视频设备管理](#) | [文档中心](#) | [声网 \(shengwang.cn\)](#)

常用场景视频回调顺序如下：

1. 指定使用某个设备：

```
setDevice(deviceId)
```

```
onLocalVideoStateChanged(this:XX, state: 1, errorCode: 0) // 1:本地视频采集设备启动成功
```

2. 使用中拔出正在使用的设备：

```
onLocalVideoStateChanged(this:XX, state: 3, errorCode: 9) //3:本地视频启动失败，9:当前正在使用的视频采集设备已经断开连接（例如，被拔出）
```

```
onVideoDeviceStateChanged(this:XX, deviceId:XX, deviceType: 3, state: 8) // deviceType 3: 视频采集设备 state 8: 设备被拔出
```

3. 重新插入之前拔出的设备，采集会自动恢复采集

```
onLocalVideoStateChanged(this: XX, state: 0, errorCode: 0) // 0:本地视频默认初始状态
```

```
onLocalVideoStateChanged(this:XX, state: 1, errorCode: 0) // 1:本地视频采集设备启动成功
```

```
onVideoDeviceStateChanged(this:XX, deviceId:XX, deviceType: 3, state: 1) // deviceType 3: 视频采集设备 state 1: 设备正在使用
```

PC端 音频设备/音频流变化回调（以Windows为例）：

onLocalAudioStateChanged[音频基础功能](#) | [文档中心](#) | [声网 \(shengwang.cn\)](#)

onAudioDeviceStateChanged[音频设备管理](#) | [文档中心](#) | [声网 \(shengwang.cn\)](#)

onAudioRoutingChanged[音频路由](#) | [文档中心](#) | [声网 \(shengwang.cn\)](#)

常用场景音频回调顺序如下：（具体参数可能不同，以实际参数为准）

1. 设备初始状态

```
onAudioDeviceStateChanged(this:XX, deviceId:XX, deviceType:0, deviceState:0) // deviceType 0: 音频播放设备 deviceState 0: 设备就绪。
```

```
onAudioDeviceStateChanged(this:XX, deviceId:XX, deviceType:1, deviceState:0) //
```

```
deviceType 1: 音频采集设备 deviceState 0: 设备就绪
```

```
onLocalAudioStateChanged(state:0, reason:0) // state:0 初始状态
```

2. 指定使用某个设备：

```
setDevice(deviceId)
```

```
onAudioDeviceStateChanged(this:XX, deviceId:XX, deviceType:0, deviceState:1) // deviceType 0: 音频播放设备 deviceState 1: 设备正在使用。
```

```
onAudioDeviceStateChanged(this:XX, deviceId:XX, deviceType:1, deviceState:1) //
```

```
deviceType 1: 音频采集设备 deviceState 1: 设备正在使用
```

```
onAudioRoutingChanged(this:XX route:0) //route:0 音频路由为带麦克风的耳机
```

```
onLocalAudioStateChanged(state:1, reason:0) // state:1 采集设备启动成功
```

```
onLocalAudioStateChanged(state:2, reason:0) // state:2 首帧编码成功
```

1. 使用中拔出正在使用的设备:

```
onAudioDeviceStateChanged(this:XX deviceId:XX, deviceType:0, deviceState:8) //  
deviceType 0: 音频播放设备 deviceState 8: 设备被拔出  
onAudioDeviceStateChanged(this:XX deviceId:XX, deviceType:1, deviceState:8) //  
deviceType 1: 音频采集设备 deviceState 8: 设备被拔出  
onAudioRoutingChanged(this:0x6000005fc718, route:4) //route:4 音频路由为外接的扬声器
```

4. 重新插入之前拔出的设备, 采集会自动恢复采集

```
onAudioDeviceStateChanged(this:XX, deviceId:XX, deviceType:0, deviceState:1) // deviceType 0: 音频播放设备 deviceState 1: 设备正在使用。  
onAudioDeviceStateChanged(this:XX, deviceId:XX, deviceType:1, deviceState:1) //  
deviceType 1: 音频采集设备 deviceState 1: 设备正在使用  
onAudioRoutingChanged(this:0x6000005fc718, route:0) //route:0 音频路由为带麦克风的耳机
```

- 在Windows设备里USB设备被认为是主板上的设备, 所以拔出后系统返回的是state = 4, 3.5mm被认为是外接设备, 拔出后返回的state = 8, 暂时无法统一; 建议在处理音频设备拔出逻辑时, deviceState4/8都要进行处理
- Mac设备 USB和3.5 mm耳机拔出的回调的state = 8

频道外:

1、视频

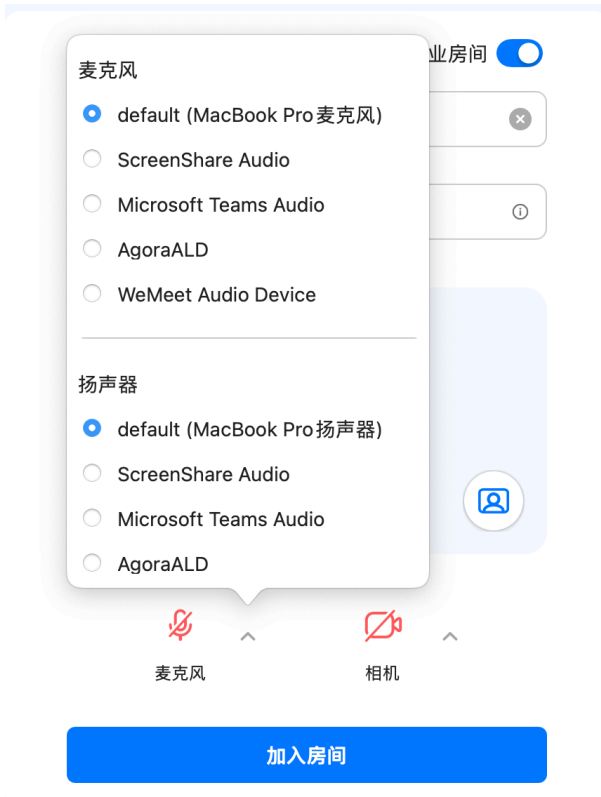
当前sdk对devicestate的监听是基于localvideoTrack的, 如果localvideoTrack没有创建成功, 则就监听不到

频道外需要调用start preview 创建video track 后可以正常收到视频设备插拔回调, 或者通过监听系统的设备插拔回调来完成UI的更新



2、音频

频道外, 音频设备插拔sdk 会有onAudioDeviceStateChanged 回调无需特殊设置



- Windows调用leavechannel 和Mac 调用enablelocaaudio(false) 后返回onAudioDeviceStateChanged(state:0), 是因为sdk state =0 即表示插入 也表示stop后的就绪, 单纯监听state= 0 回调提示新增设备会导致UI混乱, 4.4.0 版本会将插入和就绪状态分开 具体的表现为: 把设备插入这个回调, 从之前0改成了3。其他回调不变
- 连接状态:
 - 1) 非follow场景, 连接后只有一个3;
 - 2) follow场景, 连接后有3, 然后前设备的0, 然后新设备的1;
- 断开状态:
 - 1) 非follow场景, 回调5; 如果这个设备在使用中, 再回调0; 如果这个设备不在使用中, 则没有其他回调;
 - 2) follow场景, 回调5; 如果使用, 则回调0, 然后重启到新的设备的1; 如果这个设备不在使用中, 则没有其他回调;

```

320 | scope, 0};
321 |
322 | if (isDefaultDevice) {
323 |     char devName[len];
324 |
325 |     err = AudioObjectGetPropertyData(usedID, &propertyAddress, 0, NULL, &len,
326 |                                     devName);
327 |
328 |     if (err != noErr) {
329 |         RTC_LOG(LS_ERROR) << "[adm] " << __FUNCTION__ << ": " << __LINE__ << ": "
330 |         << err << "(" << FormatErrorMessage(err) << ")";
331 |         return err;
332 |     }
333 |
334 |     snprintf(name, kAdmMaxDeviceNameSize, "default (%s)", devName);
335 | } else {
336 |     if (index < deviceListLength) {
337 |         usedID = scopedDeviceIds[index];
338 |     } else {
339 |         usedID = index;
340 |     }

```

频道中

1、视频

视频插拔行为在4.2.3/4.2.2.1/前的版本行为:

- 1) 如果没有摄像头，启动视频采集后，插入摄像头，不会恢复视频采集，也没有事件回调
- 2) 如果有一个或多个摄像头，启动视频采集后，拔掉 sdk 使用的摄像头，视频采集中断。再插上任一摄像头，如果插上的是之前 sdk 使用的摄像头，会恢复视频采集，否则不会恢复视频采集
- 3) 如果有多个摄像头，启动视频采集后，拔掉非 sdk 使用的摄像头，视频采集不受影响。再插上任一摄像头，视频采集不受影响

在4.2.3/4.2.2.1/后的版本行为：

- 1) 如果没有摄像头，启动视频采集后，插入摄像头，不会恢复视频采集，但是会有事件回调
- 2) 如果有一个或多个摄像头，启动视频采集后，拔掉 sdk 使用的摄像头，视频采集中断。再插上任一摄像头，如果插上的是之前 sdk 使用的摄像头，会恢复视频采集，否则不会恢复视频采集
- 3) 如果有多个摄像头，启动视频采集后，拔掉非 sdk 使用的摄像头，视频采集不受影响。再插上任一摄像头，视频采集不受影响

Windows平台，插入新设备后，onVideoStateChanged回调新设备state:1，实际没有使用新设备，回调和实际使用设备不匹配

2、音频

1、选择系统默认

默认情况下，sdk的音频设备会跟随系统的设备变化



Mac 3.5mm耳机插拔过程会有多余的状态回调，研发判断为正常现象，建议进行去重处理

2、选择具体的某个设备后不跟随系统的默认设备变化改变

设置followSystemPlaybackDevice (false) 和followSystemRecordingDevice (false) 设置不跟随跟随系统的音频采集和播放设备 后，如果不调用setdevice 手动切换设备，将会一直使用之前选择的设备

注意：4.2.6.16之前的版本，Mac设置followSystemRecordingDevice (false) 后无法收到音频采集设备的设备插拔回调

3、虚拟声卡 (Mac: Handling Virtual Devices)：

Mac机型上在安装一些app的时候，常常会被动的安装各种虚拟声卡，主要是用来做屏幕共享。

譬如AgoraALD是声网的虚拟声卡（用于屏幕共享），Microsoft Teams Audio是安装Teams时附带安装的虚拟声卡，Loopback Audio是用户自己安装的虚拟声卡，用于单个app音频的抓取。

当我们的SDK在通信的过程中选择到了这些虚拟声卡作为采集或播放设备，可能会引起如下问题：

1. 采集或播放无声
2. 通话延迟非常大，甚至达到1s的级别
3. 回声，因为音频链路的延迟超过了aec延迟估计模块的能力（500ms）
4. 音频回调异常

因此在MacOSX SDK的实际集成过程中，一般建议如下的最佳实践：

1. **4.3.0和4.2.0版本**在onDeviceStateChanged回调中增加了设备的类型，特别标识当前使用的采集或播放设备是否是虚拟设备。其中**枚举值5代表当前使用了虚拟播放设备，枚举值6代表当前使用了虚拟采集设备**。app层收到onDeviceStateChanged回调的时候需要检查回调值，提示用户当前使用了虚拟设备，让用户决定是否需要切换相应设备。

```
/** Media device type */
typedefNS_ENUM(NSInteger, AgoraMediaDeviceType) {
    /** Unknown device */
    AgoraMediaDeviceTypeAudioUnknown = -1,
    /** Audio playback device */
    AgoraMediaDeviceTypeAudioPlayout = 0,
    /** Microphone device */
    AgoraMediaDeviceTypeAudioRecording = 1,
    /** Video render device */
    AgoraMediaDeviceTypeVideoRender = 2,
    /** Video capture device */
    AgoraMediaDeviceTypeVideoCapture = 3,
    /** Audio playback device of the app */
    AgoraMediaDeviceTypeAudioApplicationPlayout = 4,
    /** Virtual audio playback device */
    AgoraMediaDeviceTypeAudioVirtualPlayout = 5,
    /** Virtual microphone device */
    AgoraMediaDeviceTypeAudioVirtualRecording = 6,
};
```

2. **4.3.0和4.2.0版本**同样增加了两个私参，分别表示SDK不允许使用虚拟声卡作为采集或播放设备。这样SDK永远不会使用虚拟声卡作为采集和播放设备。这样设置会有一个问题，就是用户没法使用像Krisp这样的AI降噪算法，因为这个算法是通过集成到虚拟声卡里面提供的。

```
agoraKit.setParameters("{\"che.audio.allowVirtualPlayoutDevice\":false}")
agoraKit.setParameters("{\"che.audio.allowVirtualRecordingDevice\":false}")
```

一般来说建议使用上述第一种方式来集成我们的Mac SDK，让用户决定是否需要切换使用实体物理采集播放设备。

设备检测

扬声器：

开始检测：startPlaybackDeviceTest

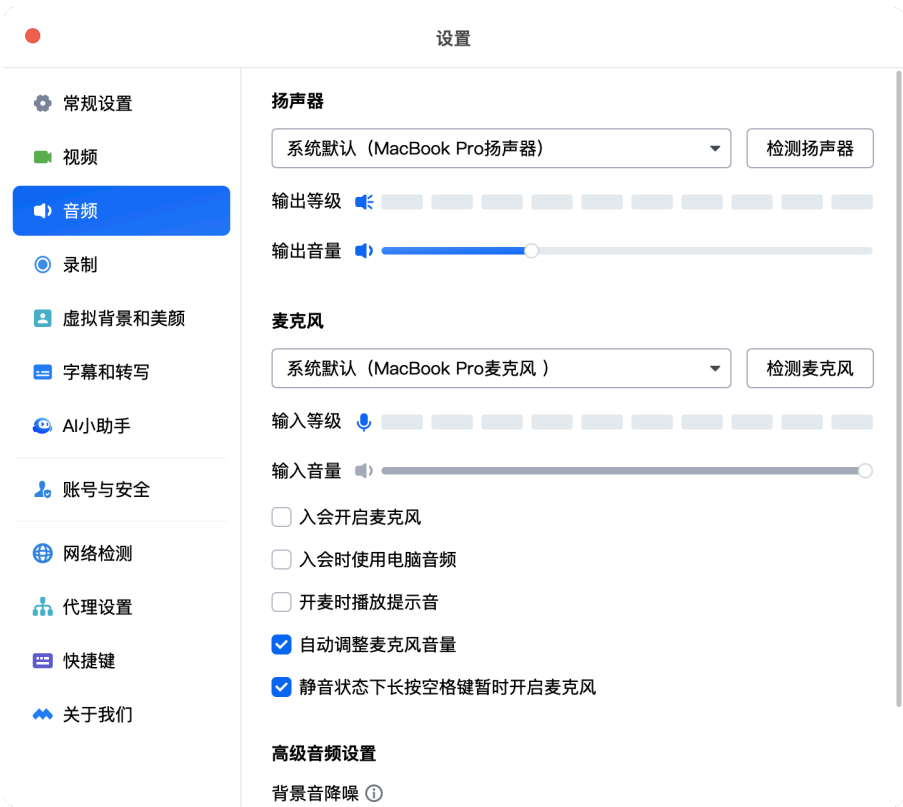
停止检测：stopPlaybackDeviceTest

麦克风：

开始检测：startRecordingDeviceTest

停止检测：stopRecordingDeviceTest

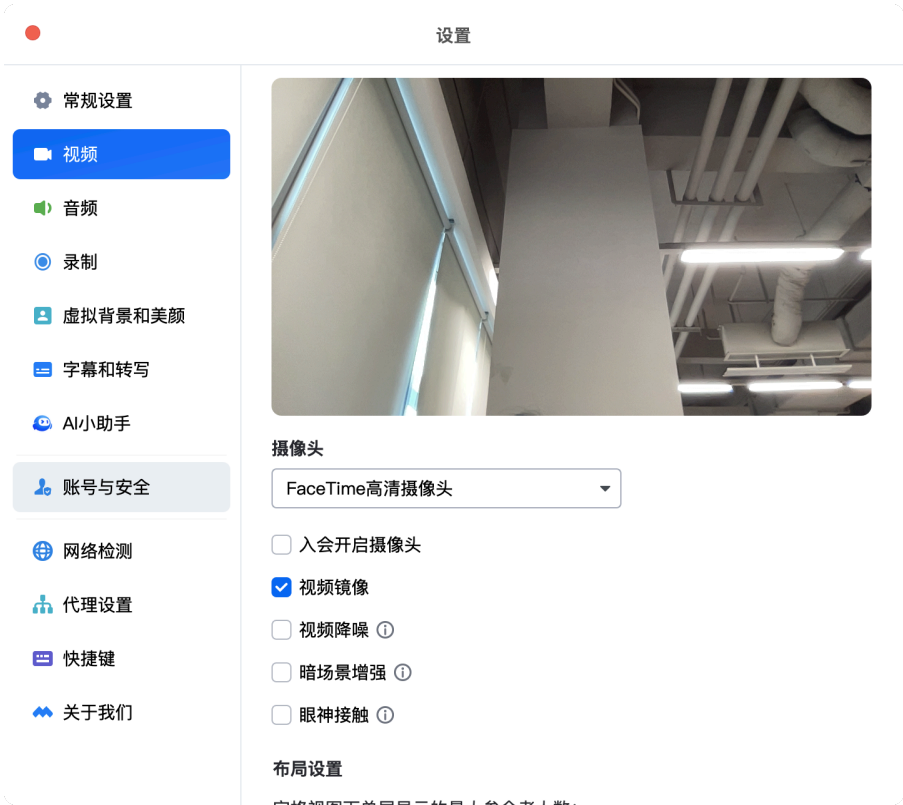
- 音量条的部分监听onAudioVolumeIndication 回调来做端上显示，扬声器的检测固定间隔100ms回调一次，麦克风的检测根据传入的参数间隔回调
- 会前检测需要在加入频道前调用对应的停止检测的 API
- 官网的文档中说明设备检测只能在加入频道前调用，但实测在目前的米哈游项目中，4.2.6 的版本支持在会中调用检测麦克风和扬声器



摄像头:

startpreview 开启会前视频预览

1. 开启预览后 直接调用 `setDevice()` 切换为其他设备, 不需要重新`startPreview()`
2. 如果拔掉正在使用的设备, 通过`setDevice()` 切换为其他设备, 需要再调用一次`startPreview()`, 否则将无法使用摄像头B开启视频预览



2. 屏幕共享

```
100 +
:81 + ATEST_F(RtcEngineExSmoke, publish_mic_and_loopback_with_different_connection) {
:82 +     // join channel with default connection, uid1 publish mic audio and camera
:83 +     agora::rtc::ChannelMediaOptions op1;
:84 +     op1.autoSubscribeVideo = true;
:85 +     op1.autoSubscribeAudio = true;
:86 +     op1.publishMicrophoneTrack = true;
:87 +     op1.publishCameraTrack = true;
:88 +     op1.enableAudioRecordingOrPlayout = true;
:89 +     op1.clientRoleType = agora::rtc::CLIENT_ROLE_BROADCASTER;
:90 +     const char* uid1 = "111";
:91 +     EXPECT_TRUE(AsyncJoinChannelWithOptions(nullptr, API_CALL_CHANNEL_NAME, uid1, op1)
:92 +         .Wait(JOIN_CALL_TIMEOUT)
:93 +         .valid);
:94 +
:95 +     // join channel with second connection, uid2 publish loopback audio and screen capture
:96 +     agora::rtc::ChannelMediaOptions op2;
:97 +     op2.autoSubscribeVideo = false;
:98 +     op2.autoSubscribeAudio = false;
:99 +     op2.publishMicrophoneTrack = false;
:100 +     op2.publishCameraTrack = false;
:101 +     op2.publishScreenTrack = true;
:102 +     op2.enableAudioRecordingOrPlayout = false;
:103 +     op2.clientRoleType = agora::rtc::CLIENT_ROLE_BROADCASTER;
:104 +     const RtcConnectionKey connection = {API_CALL_CHANNEL_NAME, 222};
:105 +     EXPECT_TRUE(
:106 +         AsyncJoinChannelExWithOptions(nullptr, connection, op2).Wait(JOIN_CALL_TIMEOUT).valid);
:107 +     EXPECT_EQ(agora::ERR_OK, engine_>enableLoopbackRecordingEx(connection.Conn(), true));
:108 +
:109 +     // default connection need to mute looback audio
:110 +     EXPECT_EQ(agora::ERR_OK, engine_>muteRemoteAudioStream(222, true));
:111 +
:112 +     EXPECT_TRUE(AsyncLeaveChannelEx(connection).Wait(API_CALL_TIMEOUT).valid);
:113 +     EXPECT_TRUE(AsyncLeaveChannel().Wait(API_CALL_TIMEOUT).valid);
:114 + }
:115 #endif
```

几个注意点:

1. 第一路enableAudioRecordingOrPlayout = true
第二路enableAudioRecordingOrPlayout = false

2. 第二路作为主播，音视频都不要订阅

3. 第一路不要订阅第二路的loopback音频，需要engine_>muteRemoteAudioStream(loopback uid, true)



共享系统声音

- enableLoopbackRecording

3. AI降噪

关闭ai降噪：

```

SetParameters(" {\che.audio.ains_mode\": 0} ");
SetParameters(" {\che-audio.nsng.lowerBound\": 80} ");
SetParameters(" {\che.audio.nsng.lowerMask\": 50} ");
SetParameters(" {\che.audio.nsng.statisticalbound\": 5} ");
SetParameters(" {\che.audio.nsng.finallowermask\": 30} ");
SetParameters(" {\che.audio.nsng.enhfactorstastical\": 200} ");
  
```

中级：

```

SetParameters(" {\che.audio.ains_mode\": 2} ");
SetParameters (" {\che.audio.nsng.lowerBound\": 80} ");
SetParameters(" {\che-audio.nsng.lowerMask\": 50} ");
SetParameters (" {\che.audio.nsng.statisticalbound\": 5} ");
SetParameters(" {\che.audio.nsng.finallowermask\": 30} ");
SetParameters (" {\che.audio.nsng.enhfactorstastical\": 200} ");
  
```

高：

```

SetParameters(" {\che.audio.ains_mode\": 2} ");
  
```

```

SetParameters ("{\"che.audio.nsg.lowerBound\": 10}");

SetParameters ("{\"che-audio.nsg.lowerMask\": 10}");

SetParameters ("{\"che.audio.nsg.statisticalbound\": 0}");

SetParameters ("{\"che.audio.nsg.finallowermask\": 8}");

SetParameters ("{\"che.audio.nsg.enhfactorstastical\": 200}");

```

Dump音频数据

- Predump
 1. 加入频道前: 参考- {"che.audio.enable.predump":{"enable":"true","duration":"60"}}
 2. 加入频道后: {"che.audio.dump_path":"/tmp"} {"che.audio.start.predump":true}
- 开启Dump


```

{"rtc.debug.enable":true}

{"che.audio.apm_dump":true}

```

警告/错误码处理指南

- [native最佳实践错误码集成](#)

其他需要注意的点:

- 1、windows硬件静音麦克风的问题:
 - a): 加入频道前用 `audio_manager->get()->getRecordingDeviceMute(mute)` 获取设备是否静音, 之后看是否需要调用`setRecordingDeviceMute(false)`来打开采集,
 - b): 频道前不检查状态, 加入频道后用户unmute, 直接调用一下`setRecordingDeviceMute(false)`来打开采集, 确认采集是打开的
- 2) 频道内开关硬件麦克风, sdk会回调onAudioDeviceVolumeChanged, 之后调用`setRecordingDeviceMute(false)`来打开采集
- 2、大重构版本 不能在onRenderVideoFrame 里面调用了 `setRemoteVideoStreamType`, 会导致的死锁。
- 3、`engine.setParameters("{\"che.audio.force_bluetooth_a2dp\":true}");` 解决蓝牙协议不匹配的问题。
- 4、支持啸叫检测

使用方式:

加入频道前设置啸叫检测模式:

```
self.agoraKit.setHowlingControlMode(true, mode: AgoraAudioHcMode.detect, activity: AgoraAudioHcActivity.high, thresh: 2)
```

在回调中接收, 啸叫的帧数, 基本上可以认为 收到的了帧数count 回调, 代表当前发生了啸叫。

```

func rtcEngine(_ engine: AgoraRtcEngineKit, howlingStatus howlingCount: Int32) {
    LogUtils.log(message: "jxztest :howlingCount:\(howlingCount).", level: .info)
}

```

- 5、android 的屏幕共享一般就是两个回调一个是state=LOCAL_VIDEO_STREAM_STATE_ENCODING, error=ERR_OK, 一个是state=LOCAL_AUDIO_STREAM_STATE_FAILED, error=ERR_SCREEN_CAPTURE_SYSTEM_NOT_SUPPORTED